COMPUTER ORGANIZATION & DESIGN

(20IC209P)

CPU Project

Submitted by

Milav Patel	**23BIT190**
Naitik Jadav	**23BIT191**
Chirag Maloo	**23BIT192**
Sumit Mali	**23BIT193**
Jwanil Modi	**23BIT194**

Department : ICT (Div-3)

Submitted to

Dr. Pallab Nath

Table of Content

Sr.no	Title	Pg.no
1	Aim	1
2	List of Figures	2
3	Table of Circuits	3
4	Assumptions & Instruments required	4
5	Working Principle	5
6	Circuits	11
7	Conclusion	25

Aim : To design a 16-bit Central Processing Unit (CPU) using Logisim.

Tasks :

a) To design various 16-bit & 12-bit registers.

b) To design Common Bus.

c) To design 16-bit ALU.

d) To design Sequence Counter.

e) To design Control Unit.

f) To design Encoder Logic for common bus and ALU.

g) To understand working principle of CPU.

h) To Integrate & Verify the working of the CPU.

List of Figures

Figure.no	Description	Pg.no
1	Lists of registers	6
2	Lists of operations	7
3	List of Register Reference Instructions	9
4	List of Memory Reference Instructions	9
5	Complete Instruction Set	10

Table of Circuits

Sr.no	Circuit
1	Timing Signal Generator
2	Sequence Counter
3	Data Register
4	Instruction Register
5	Accumulator
6	Temporary Register
7	Address Register
8	Program Counter
9	A.L.U
10	Common Bus
11	Common Bus Encoder
12	A.L.U Encoder
13	Input-Output Logic of Common Bus
14	E flip flop
15	Extra Circuit
16	Main Circuit
17	Memory

Assumptions and instruments required

- Assumptions:

a) Implementation of Interrupt is neglected.

b) Implementation of Input-Output Instruction is neglected.

- Instruments Required for CPU Design:

a) Memory (RAM): of size 4096x16.

b) Registers: AR, PC, DR, AC, IR, TR.

c) Flip-flops: E, I

d) Decoders: a 3x8 opcode decoder 4x16 timing decoder.

e) A 16-bit common bus with ALU integrated in it.

f) ALU and Common Bus Encoder.

g) A 4-bit sequence counter.

h) Control Unit

Working principle

- Computer instructions are binary codes that represent a set of micro-operations and are stored in memory with data. The computer reads each instruction and puts it in the control register, which interprets the binary code and executes the instruction by operating the micro. Each computer has its own specific instructions, and the concept of storage, that is, the ability to store and execute instructions, is the main symbol of the general computer.
- Command numbers are binary numbers that instruct the computer to perform a specific operation, such as addition, subtraction, multiplication, or addition. Function codes are part of the expression that defines functions. The control unit receives instructions from memory, interprets op-codes, and sets control signals to initiate

  micro-operations in the computer register. Each op-code contains a specific set of micro-transactions required for the hardware implementation of the transaction.
- Computer commands are stored in memory and executed sequentially. The control unit reads an instruction from a specific memory location and executes it, then moves on to the next instruction in the sequence. A counter is required to keep track of the memory address of the next instruction. In addition, the control unit needs a register to store instructions

after reading of memory and a register to manage data and hold memory addresses.

The registers are as follows with their functions:

Register symbol	Number of bits	Register name	Function
DR	16	Data register	Holds memory operand
AR	12	Address register	Holds address for memory
AC	16	Accumulator	Processor register
IR	16	Instruction register	Holds instruction code
PC	12	Program counter	Holds address of instruction
TR	16	Temporary register	Holds temporary data
INPR	8	Input register	Holds input character
OUTR	8	Output register	Holds output character

Figure 1 : List of Registers

- A basic computer has a control unit, eight registers, and a storage unit. Use a mixed bus to properly transfer data between them. The difference between binary options determines which output is placed on the bus. Registers and memory are connected to the bus via input/output lines, and data is transferred as the clock changes. The registers and outputs are 8 bits each, the other registers are 12 or 16 bits.
- Five registers have three controls (load, boost, clear), while two have only one LD entry. The system has an address register to specify the memory location and a bus for data memory I/O. The accumulator register has 16 inputs from the adder and logic circuits for various operations and to receive data from the data and input registers. The clock switch at the end of the loop transfers the data to the register and various operations can be performed in a loop by placing the data on the bus, allowing the join list to be entered.

- A simple computer has three types of 16-bit instructions. The opcode portion is three bits and the remaining 13 bits depend on the opcode encountered. The memory instruction uses a 12-bit address and one bit for

type I addressing. Bit 15 of opcode III for instruction register is set to 0 and indicates an AC register operation or test. For IO instruction, bit 15 of Opcode III is set to 1 and indicates the type of IO operation or test. The teach mode is defined by four sections in positions 12 to 15 of the computer control.

- The total number of instructions is 25, which exceeds the maximum of eight different functions due to the use of residuals in registers and input instructions.
- The hexadecimal number of the command is equal to the hexadecimal value of its binary code. Each hexadecimal number represents four things. The memory instruction contains a 12-bit address represented by three hexadecimal digits with the last lower case i. When I = 0, the last four digits have hexadecimal values from 0 to 6; When I = 1, the last four have values from 8 to E.
- The register using the instruction has 0111 constant four (hex 7) and the remaining three bits represent the binary equivalent of 12 bits. The I/O instruction uses 16 bits, the last four of which are always 1111 (hex F).

The following figure explains everything in easy manner:

9

	Hexadecimal code		
Symbol	$I = 0$	$I = 1$	Description
AND	0xxx	8xxx	AND memory word to *AC*
ADD	1xxx	9xxx	Add memory word to *AC*
LDA	2xxx	Axxx	Load memory word to *AC*
STA	3xxx	Bxxx	Store content of *AC* in memory
BUN	4xxx	Cxxx	Branch unconditionally
BSA	5xxx	Dxxx	Branch and save return address
ISZ	6xxx	Exxx	Increment and skip if zero

CLA	7800	Clear *AC*
CLE	7400	Clear *E*
CMA	7200	Complement *AC*
CME	7100	Complement *E*
CIR	7080	Circulate right *AC* and *E*
CIL	7040	Circulate left *AC* and *E*
INC	7020	Increment *AC*
SPA	7010	Skip next instruction if *AC* positive
SNA	7008	Skip next instruction if *AC* negative
SZA	7004	Skip next instruction if *AC* zero
SZE	7002	Skip next instruction if *E* is 0
HLT	7001	Halt computer

Figure 2 : List of Operations

- A computer's instructions should include arithmetic, logic, and function commands, data move commands, function control commands, and input/output commands. Subcommands that provide all these functions include ADD, AND, LDA, and STA.
- The timing of all registers in the computer is controlled by a clock generator that uses clock pulses for all the flip-flops and registers, including the control room. Manage the signals generated by the control unit for the list to change state.
- Execution of registers using commands is completed during T3, and the SC counter is cleared. The first seven instructions

  Focus on clear, complement, rotate, increment microoperations in the AC or E register. The next four commands in sequence cause the next command to be skipped when conditions are specified. The HLT command removes the start-stop start S so that the counter sequence is not counted.

  The control functions and microoperations for these instructions are listed in figure below:

$D_7I'T_3 = r$ (common to all register-reference instructions)
$IR(i) = B_i$ [bit in $IR(0–11)$ that specifies the operation]

	r:	$SC \leftarrow 0$	Clear SC
CLA	rB_{11}:	$AC \leftarrow 0$	Clear AC
CLE	rB_{10}:	$E \leftarrow 0$	Clear E
CMA	rB_9:	$AC \leftarrow \overline{AC}$	Complement AC
CME	rB_8:	$E \leftarrow \overline{E}$	Complement E
CIR	rB_7:	$AC \leftarrow \text{shr } AC,\ AC(15) \leftarrow E,\ E \leftarrow AC(0)$	Circulate right
CIL	rB_6:	$AC \leftarrow \text{shl } AC,\ AC(0) \leftarrow E,\ E \leftarrow AC(15)$	Circulate left
INC	rB_5:	$AC \leftarrow AC + 1$	Increment AC
SPA	rB_4:	If $(AC(15) = 0)$ then $(PC \leftarrow PC + 1)$	Skip if positive
SNA	rB_3:	If $(AC(15) = 1)$ then $(PC \leftarrow PC + 1)$	Skip if negative
SZA	rB_2:	If $(AC = 0)$ then $PC \leftarrow PC + 1)$	Skip if AC zero
SZE	rB_1:	If $(E = 0)$ then $(PC \leftarrow PC + 1)$	Skip if E zero
HLT	rB_0:	$S \leftarrow 0$ (S is a start–stop flip-flop)	Halt computer

Figure 3 : List of Register Reference Instruction

- The precise function of memory-reference instructions requires defining the microoperations needed for their execution. Figure below lists seven such instructions, with decoded outputs and effective addresses. The symbolic description of each instruction is given in terms of register transfer notation, with a sequence of microoperations required for actual execution due to the need to read data from memory to a register.

Symbol	Operation decoder	Symbolic description
AND	D_0	$AC \leftarrow AC \wedge M[AR]$
ADD	D_1	$AC \leftarrow AC + M[AR],\quad E \leftarrow C_{out}$
LDA	D_2	$AC \leftarrow M[AR]$
STA	D_3	$M[AR] \leftarrow AC$
BUN	D_4	$PC \leftarrow AR$
BSA	D_5	$M[AR] \leftarrow PC,\quad PC \leftarrow AR + 1$
ISZ	D_6	$M[AR] \leftarrow M[AR] + 1,$ If $M[AR] + 1 = 0$ then $PC \leftarrow PC + 1$

Figure 4 : List of Memory Reference Instruction

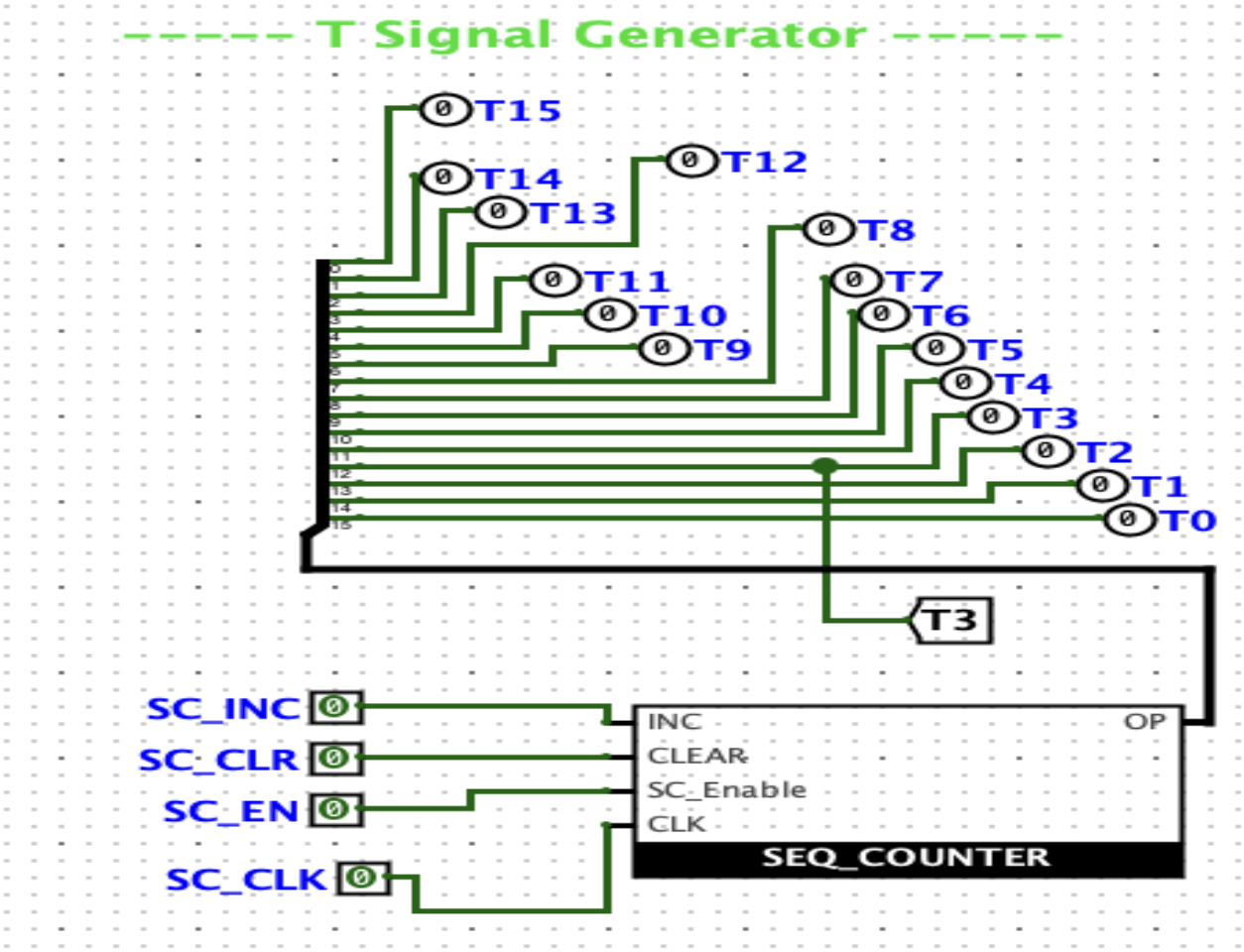
Following are the set of operations that were performed and analysed in CPU:

Fetch	$R'T_0$:	$AR \leftarrow PC$
	$R'T_1$:	$IR \leftarrow M[AR], \quad PC \leftarrow PC + 1$
Decode	$R'T_2$:	$D_0, \ldots, D_7 \leftarrow$ Decode $IR(12\text{–}14)$,
		$AR \leftarrow IR(0\text{–}11), \quad I \leftarrow IR(15)$
Indirect	$D_7'IT_3$:	$AR \leftarrow M[AR]$
Interrupt:		
$T_0'T_1'T_2'(IEN)(FGI + FGO)$:		$R \leftarrow 1$
	RT_0:	$AR \leftarrow 0, \quad TR \leftarrow PC$
	RT_1:	$M[AR] \leftarrow TR, \quad PC \leftarrow 0$
	RT_2:	$PC \leftarrow PC + 1, \quad IEN \leftarrow 0, \quad R \leftarrow 0, \quad SC \leftarrow 0$
Memory-reference:		
AND	D_0T_4:	$DR \leftarrow M[AR]$
	D_0T_5:	$AC \leftarrow AC \wedge DR, \quad SC \leftarrow 0$
ADD	D_1T_4:	$DR \leftarrow M[AR]$
	D_1T_5:	$AC \leftarrow AC + DR, \quad E \leftarrow C_{out}, \quad SC \leftarrow 0$
LDA	D_2T_4:	$DR \leftarrow M[AR]$
	D_2T_5:	$AC \leftarrow DR, \quad SC \leftarrow 0$
STA	D_3T_4:	$M[AR] \leftarrow AC, \quad SC \leftarrow 0$
BUN	D_4T_4:	$PC \leftarrow AR, \quad SC \leftarrow 0$
BSA	D_5T_4:	$M[AR] \leftarrow PC, \quad AR \leftarrow AR + 1$
	D_5T_5:	$PC \leftarrow AR, \quad SC \leftarrow 0$
ISZ	D_6T_4:	$DR \leftarrow M[AR]$
	D_6T_5:	$DR \leftarrow DR + 1$
	D_6T_6:	$M[AR] \leftarrow DR$, if $(DR = 0)$ then $(PC \leftarrow PC + 1), \quad SC \leftarrow 0$
Register-reference:		
		$D_7I'T_3 = r$ (common to all register-reference instructions)
		$IR(i) = B_i \ (i = 0, 1, 2, \ldots, 11)$
	r:	$SC \leftarrow 0$
CLA	rB_{11}:	$AC \leftarrow 0$
CLE	rB_{10}:	$E \leftarrow 0$
CMA	rB_9:	$AC \leftarrow \overline{AC}$
CME	rB_8:	$E \leftarrow \overline{E}$
CIR	rB_7:	$AC \leftarrow$ shr $AC, \quad AC(15) \leftarrow E, \quad E \leftarrow AC(0)$
CIL	rB_6:	$AC \leftarrow$ shl $AC, \quad AC(0) \leftarrow E, \quad E \leftarrow AC(15)$
INC	rB_5:	$AC \leftarrow AC + 1$
SPA	rB_4:	If $(AC(15) = 0)$ then $(PC \leftarrow PC + 1)$
SNA	rB_3:	If $(AC(15) = 1)$ then $(PC \leftarrow PC + 1)$
SZA	rB_2:	If $(AC = 0)$ then $PC \leftarrow PC + 1)$
SZE	rB_1:	If $(E = 0)$ then $(PC \leftarrow PC + 1)$
HLT	rB_0:	$S \leftarrow 0$

Figure 5 : Complete Instruction Set

Circuits

1. Timing Signal Generator

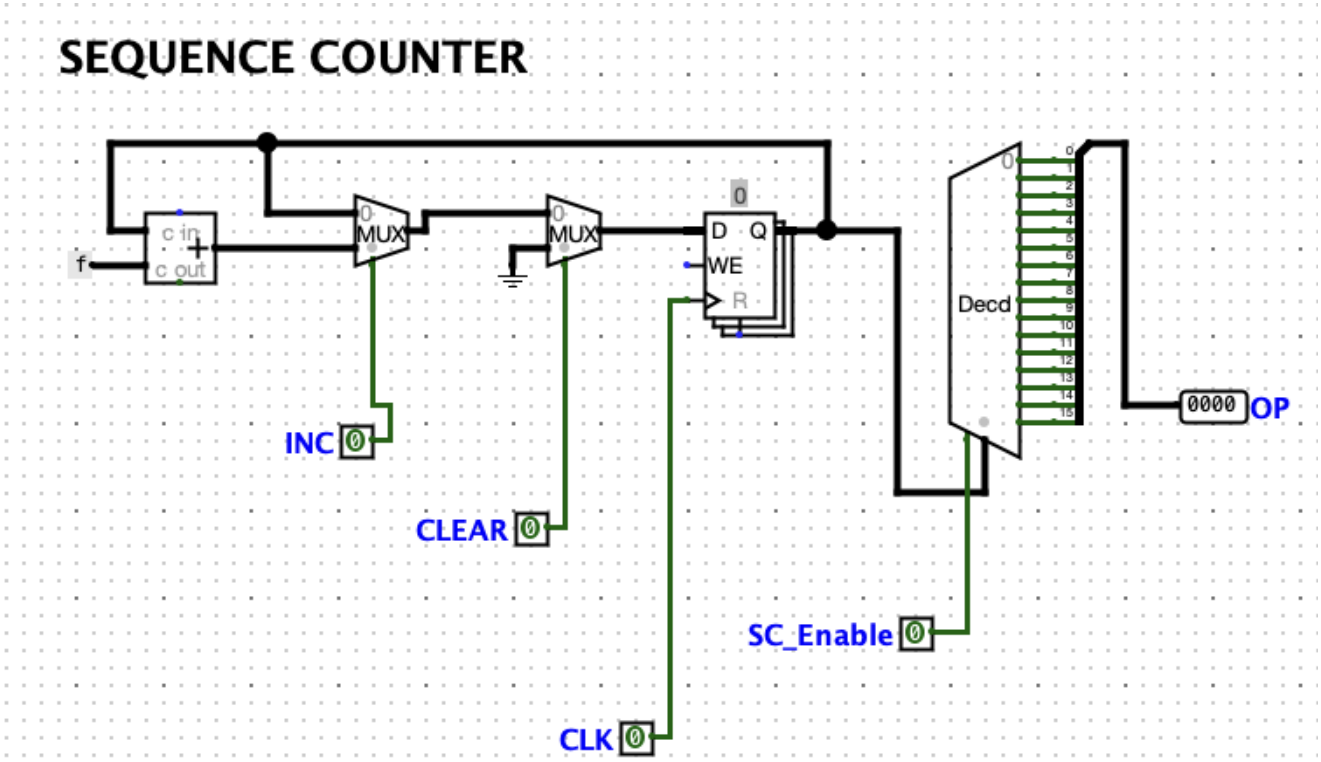


Circuit-1: Timing Signal Generator

Explanation:

This circuit is use to generate timing signals for control unit

2. Sequence Counter.

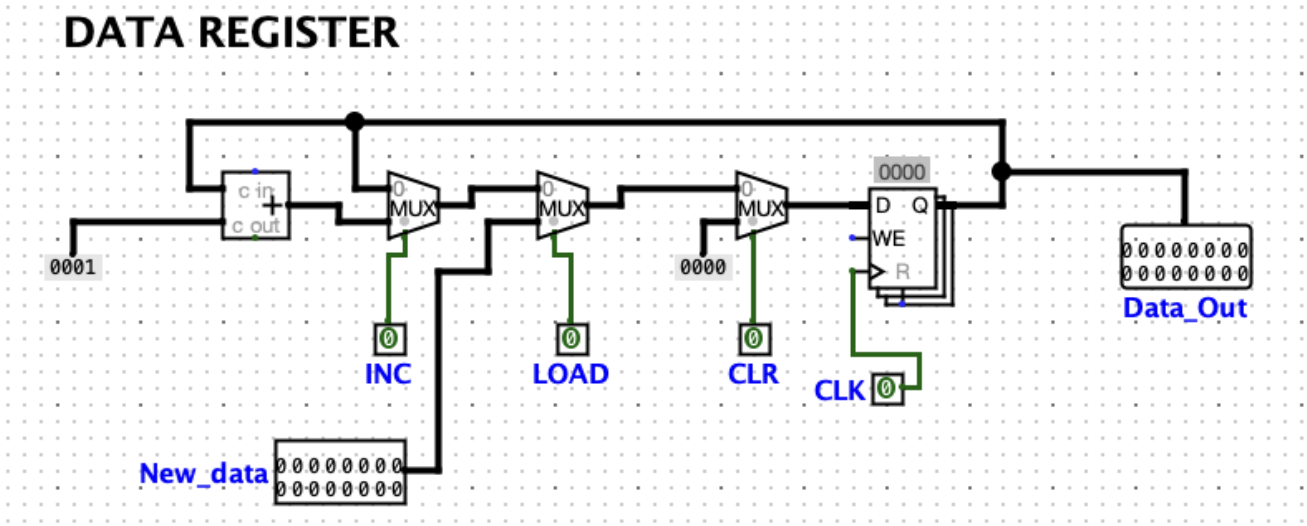


Circuit-2: Internal Circuit of Sequence Counter

Explanation:

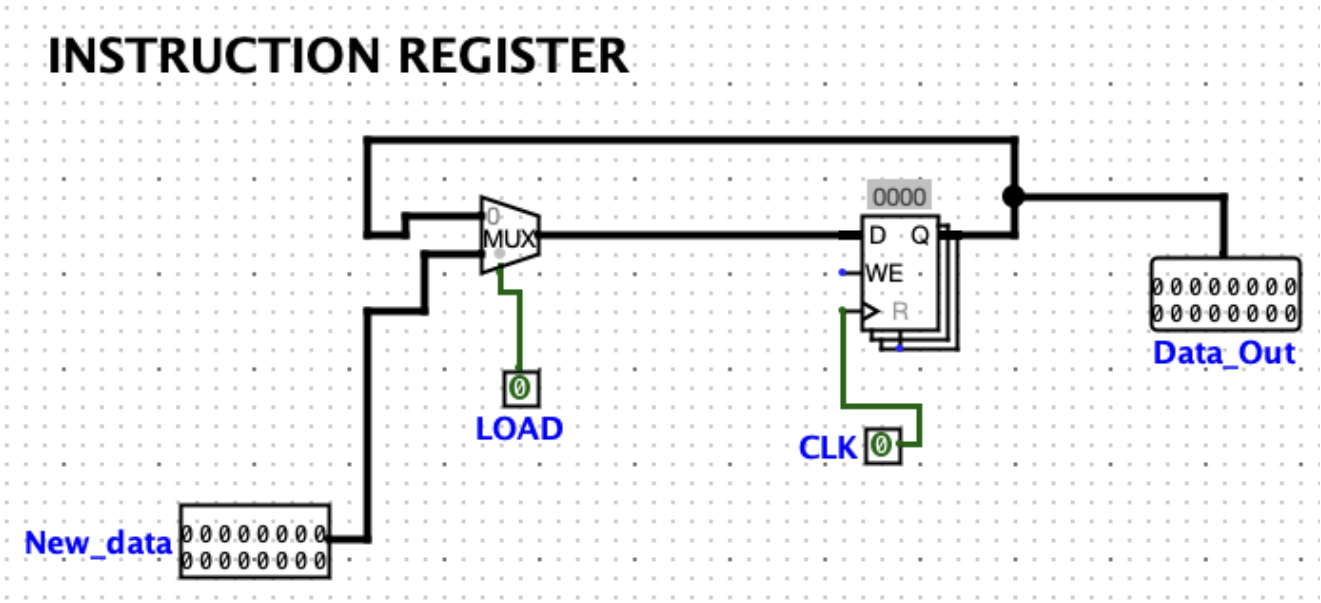
This circuit was the most crucial circuit of sequence counter. This circuit is used to control time signals. The circuit provides user the choice to control his/her next instruction.

3. Data Register:



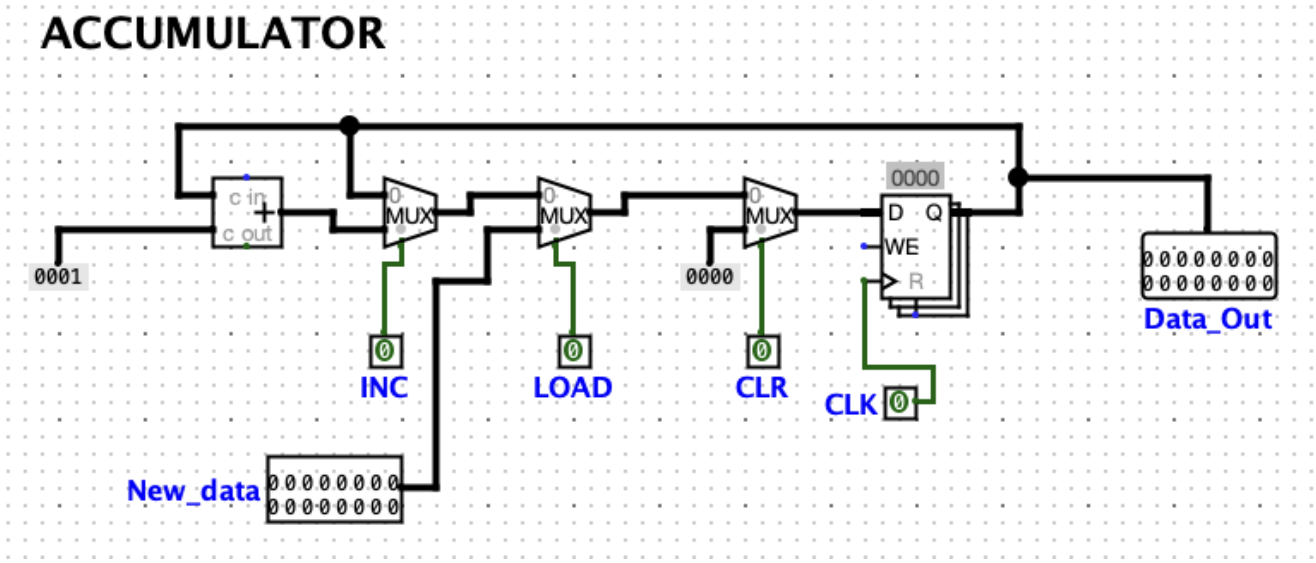
Circuit-3: Data Register

4. Instruction Register:



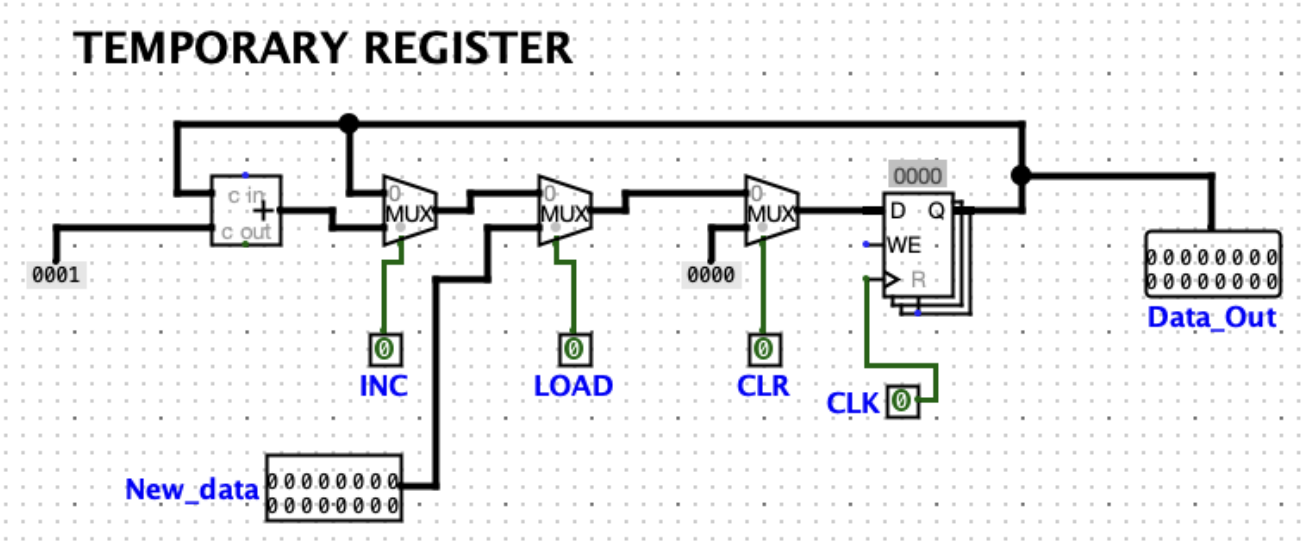
Circuit-4: Instruction Register

5. Accumulator:



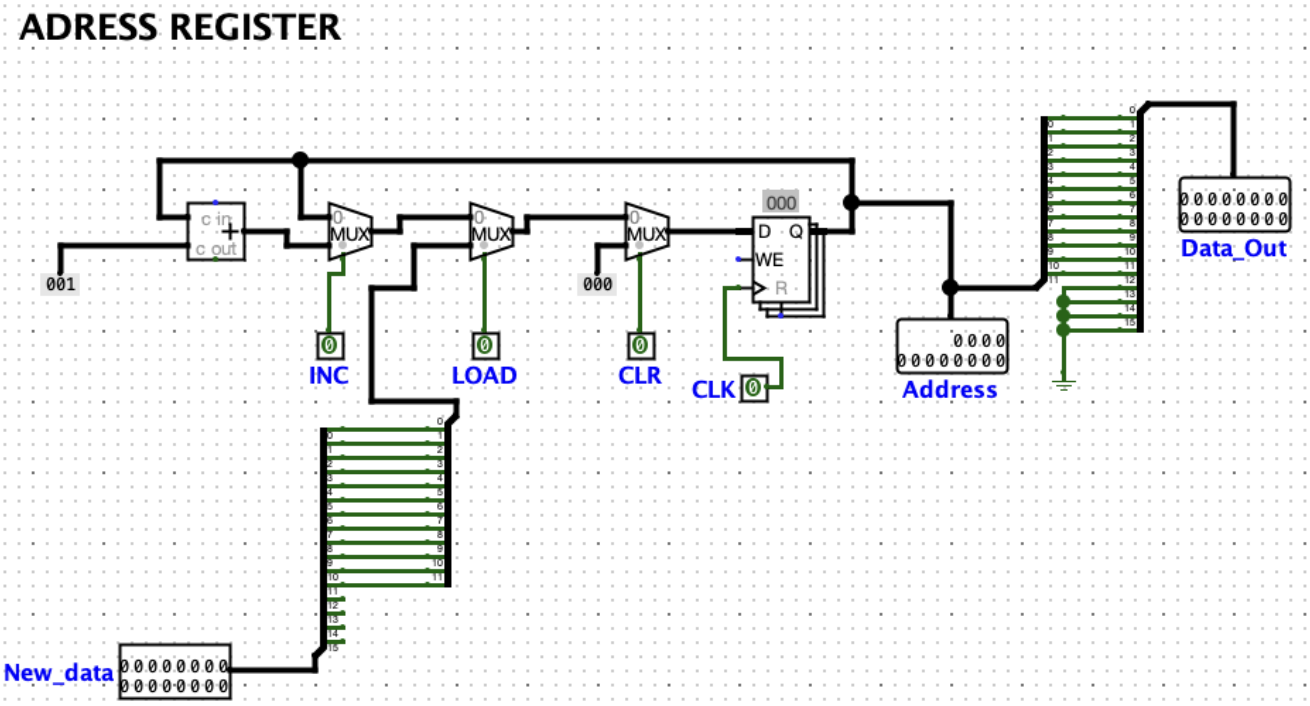
Circuit-5: Accumulator

6. Temporary Register:



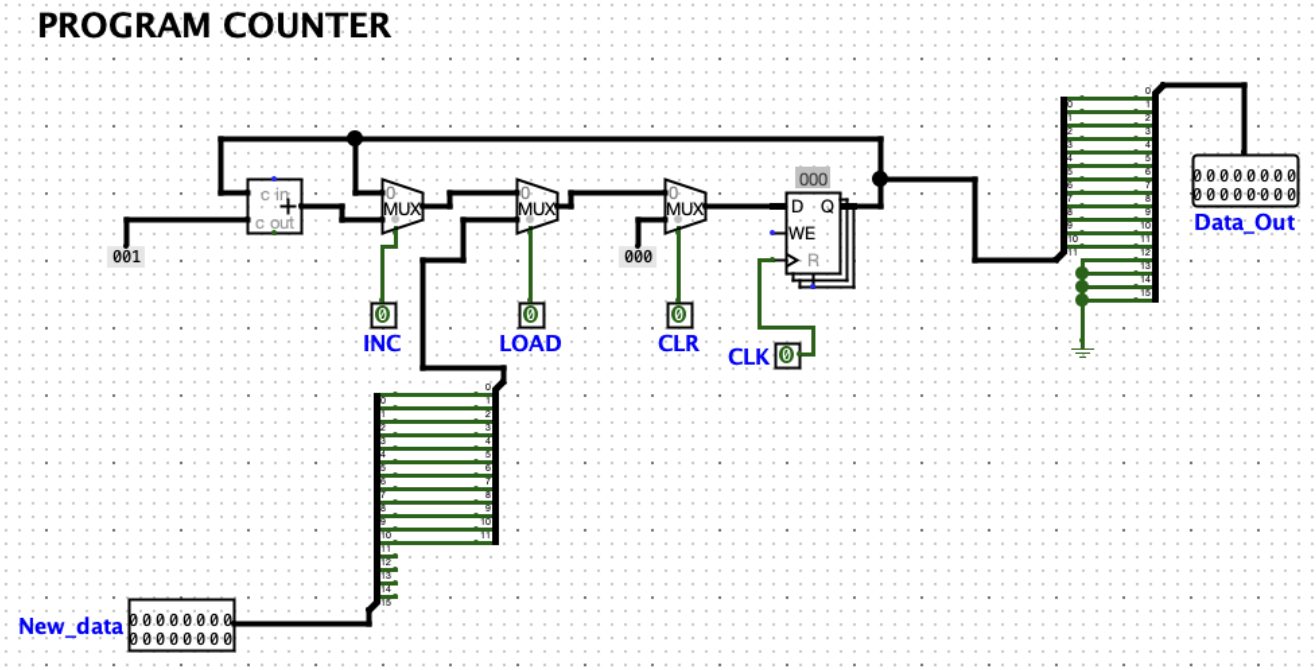
Circuit-6: Temporary Register

7. Address Register:



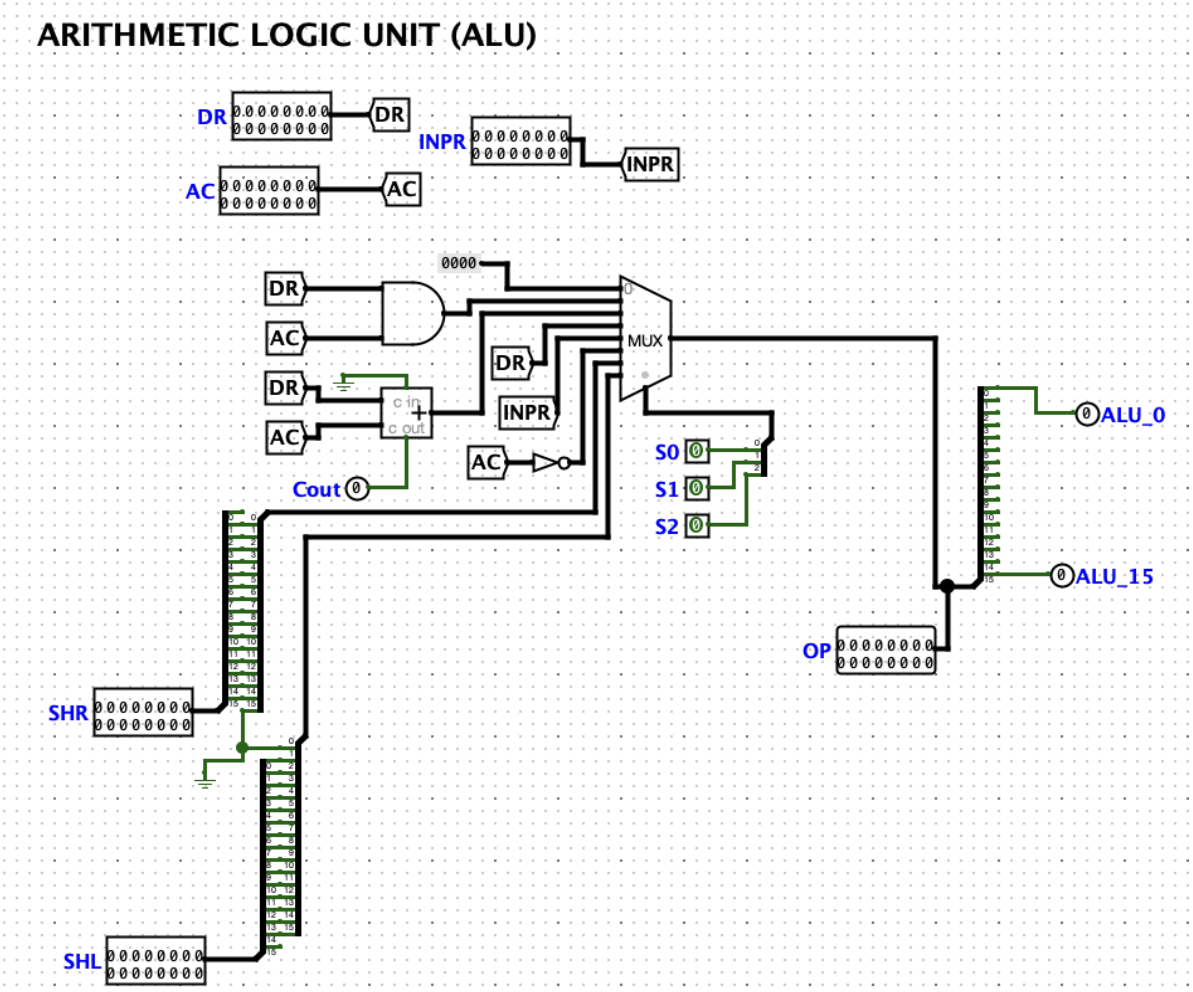
Circuit-7: Address Register

8. Program Counter



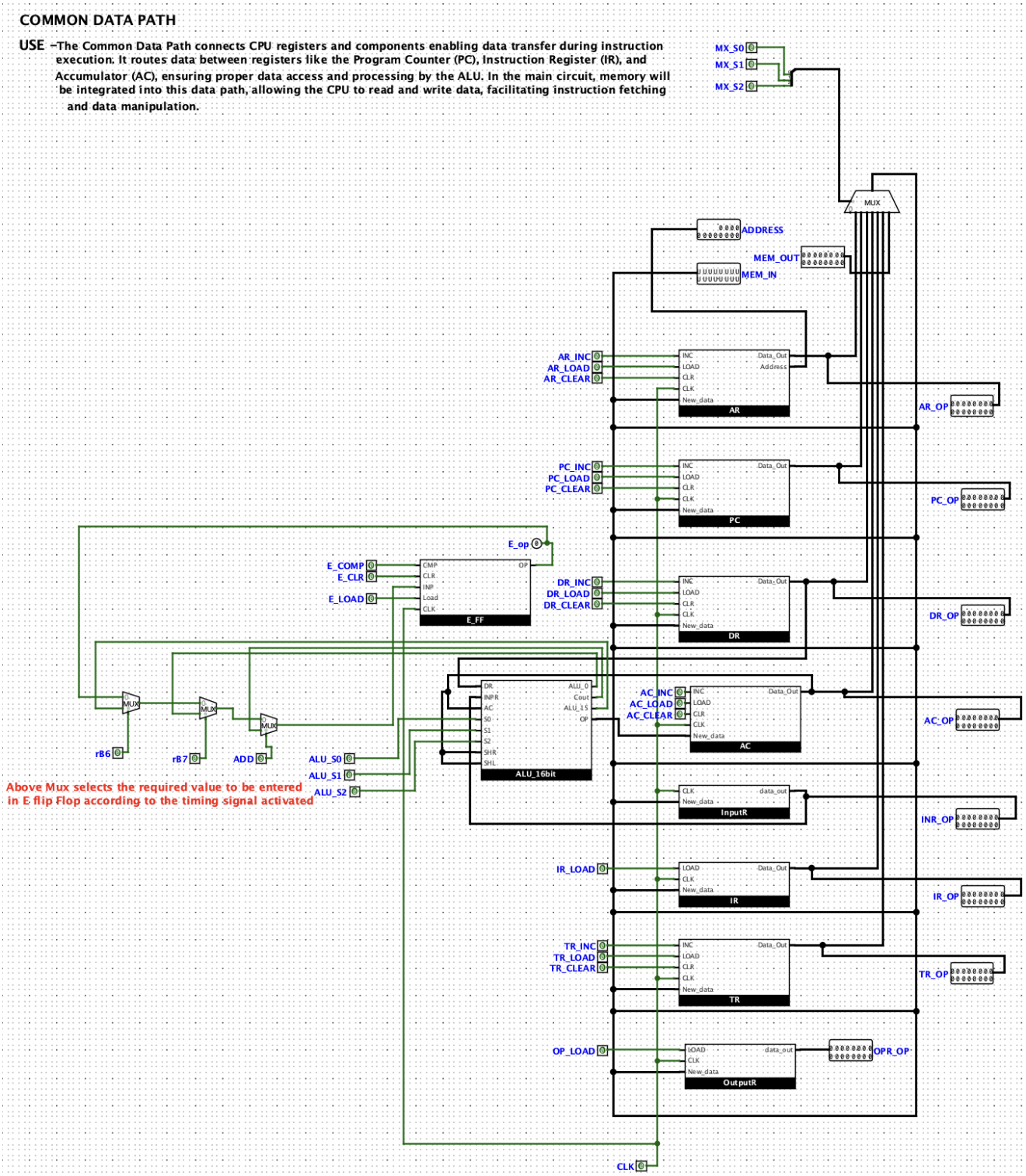
Circuit-8: Program Counter

9. ALU.:



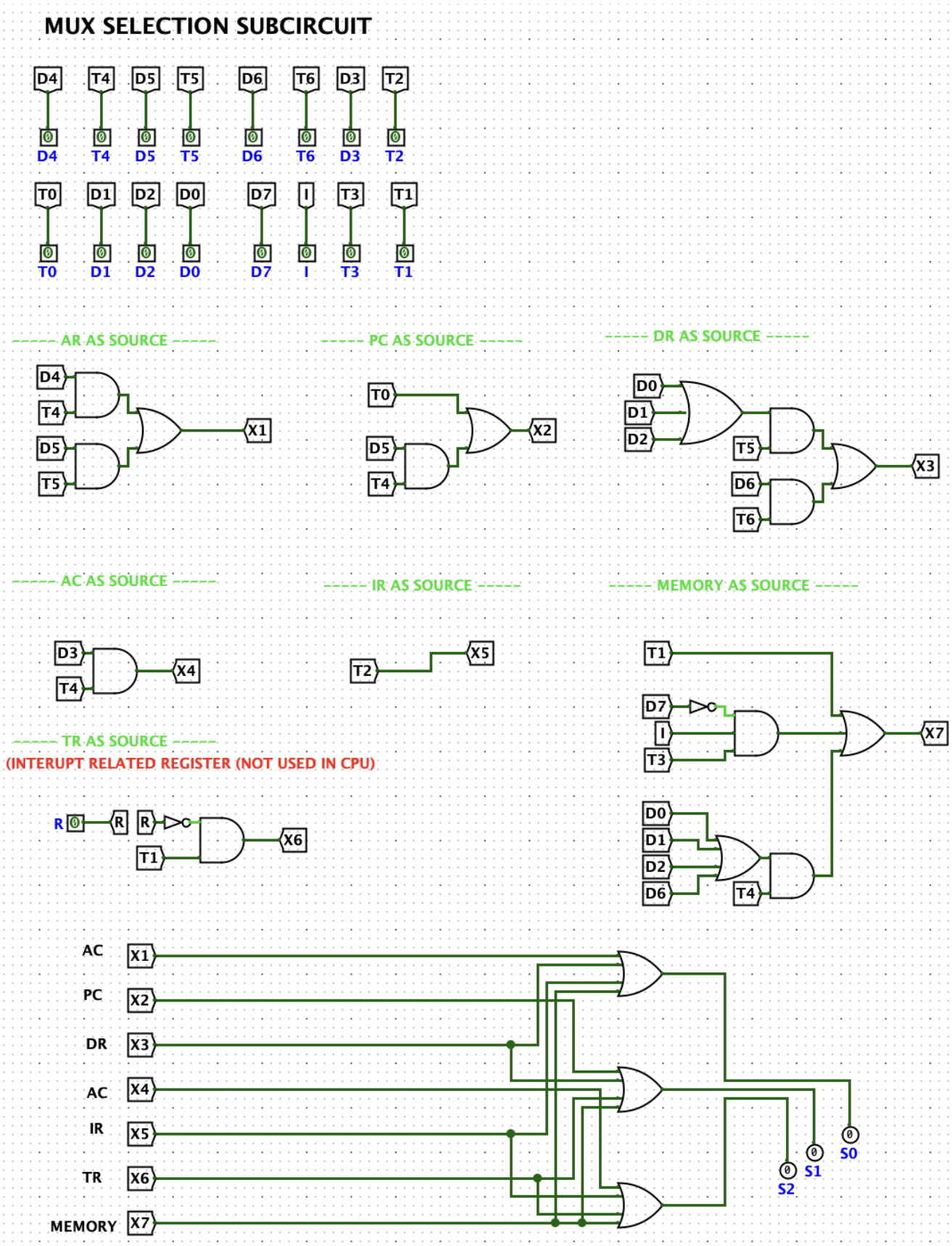
Circuit-9: ALU

10. Common Bus:



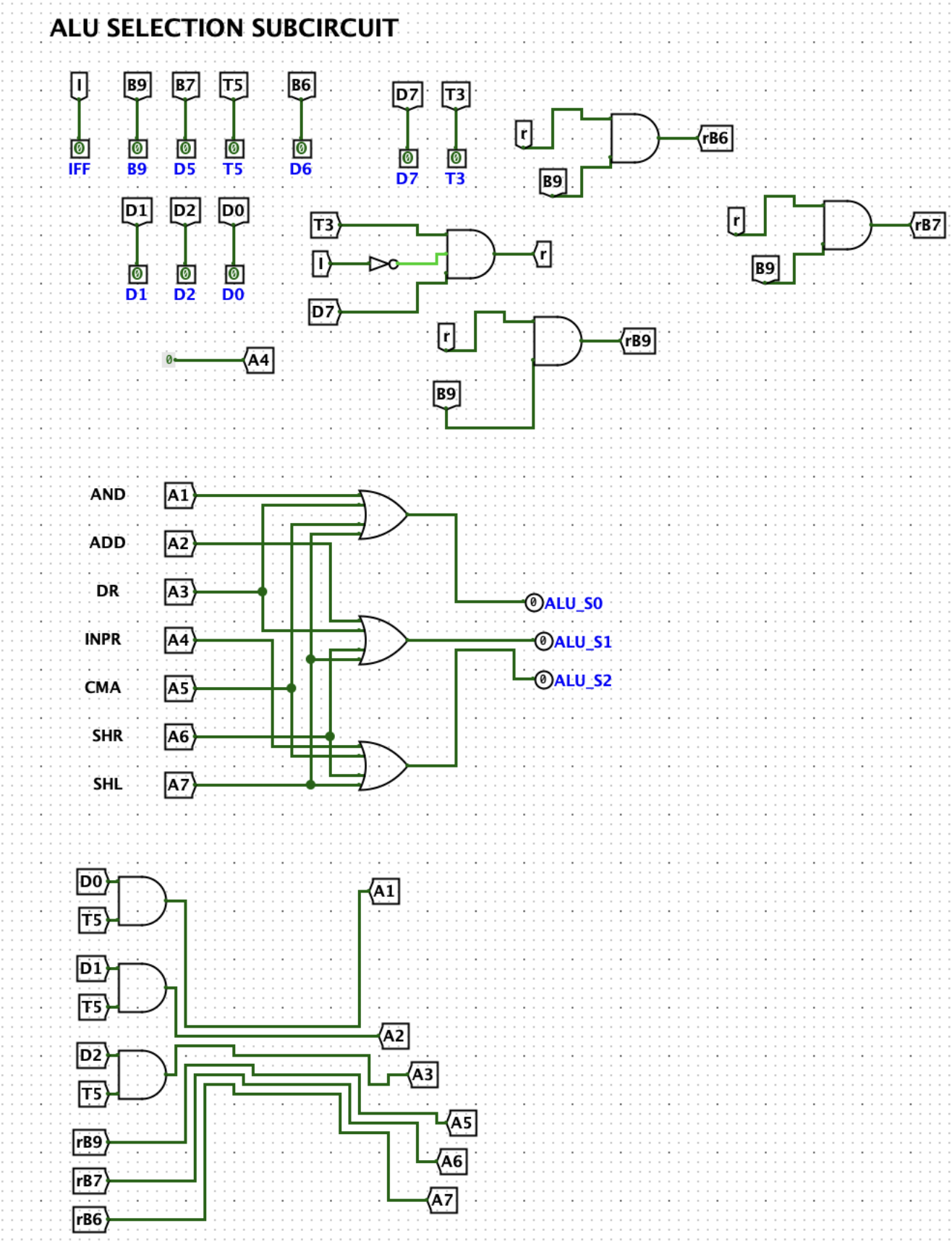
Circuit-10: Common BUS

11. Common Bus Encoder:



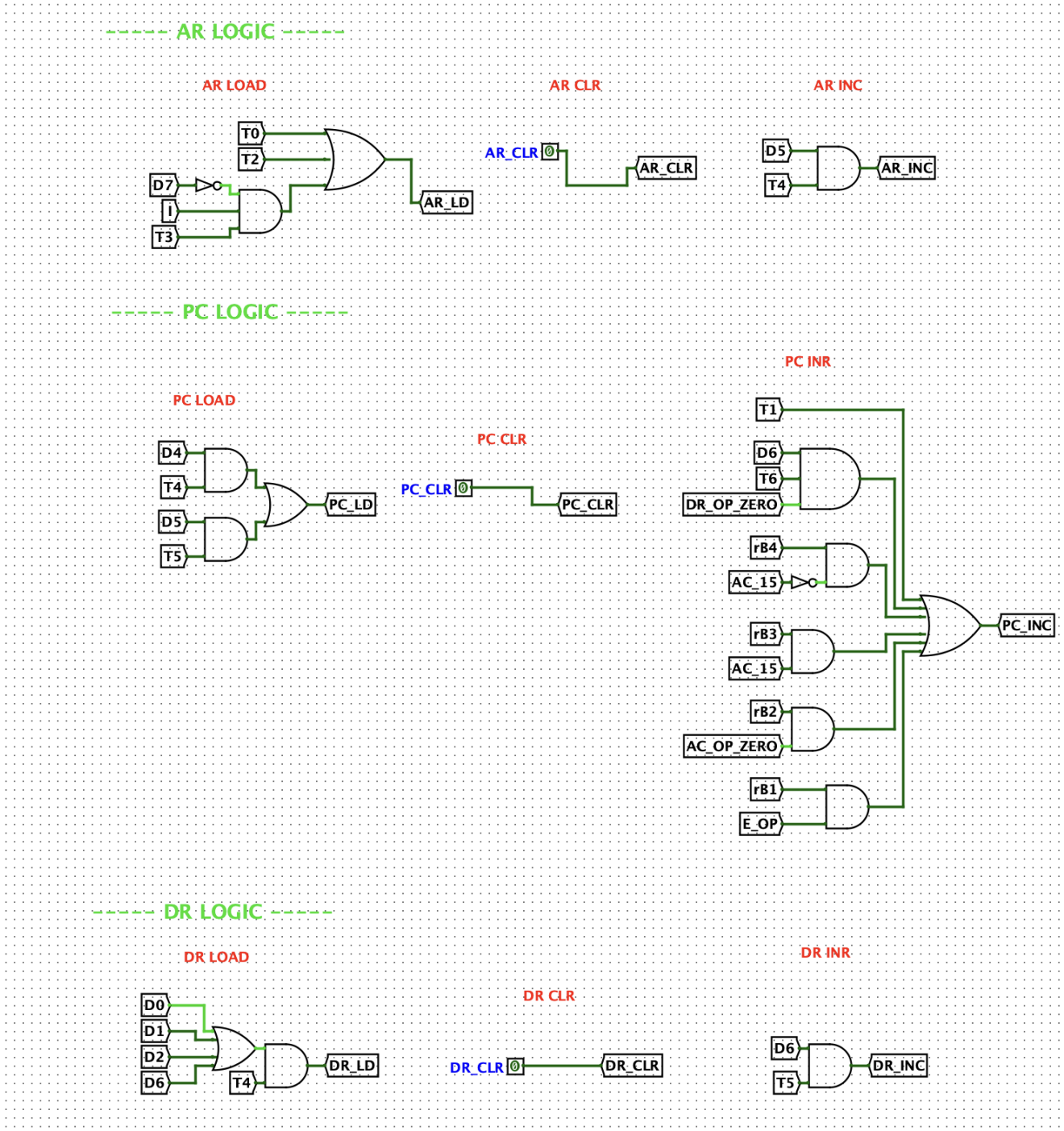
Circuit-11: Common BUS Encoder

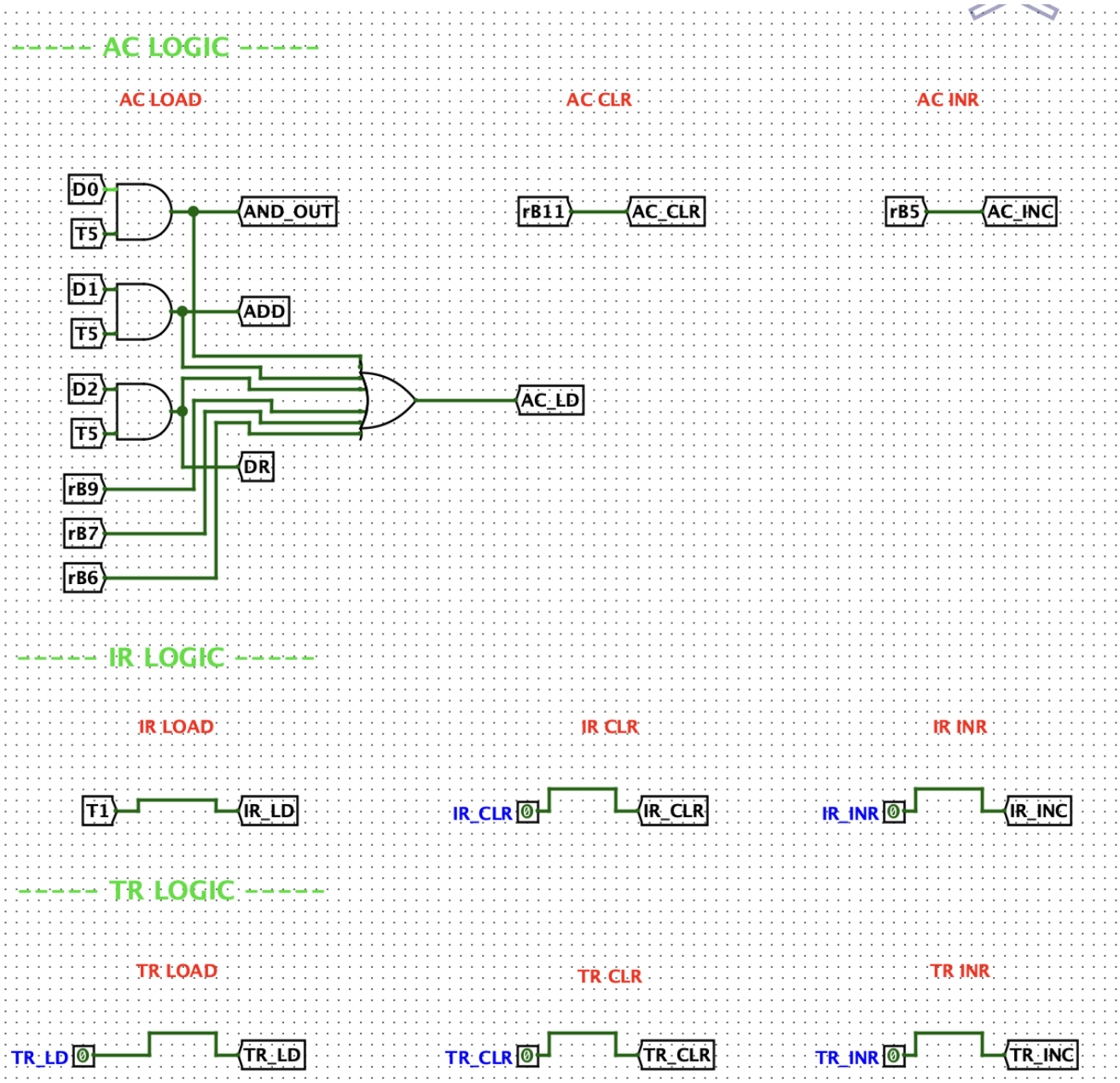
12. ALU Encoder:

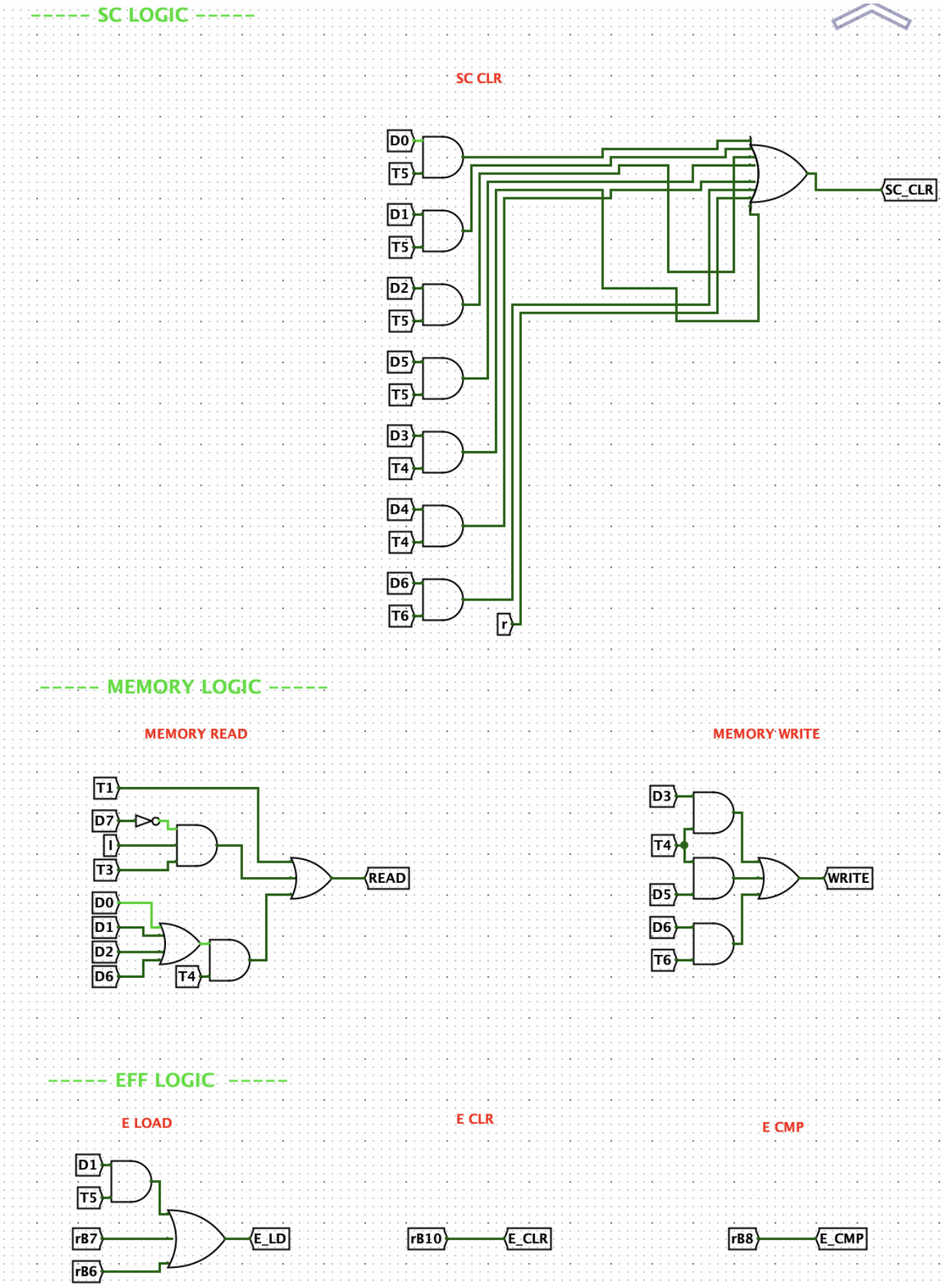


Circuit-12: ALU Encoder

13. Input-Output Logic of Common Bus:

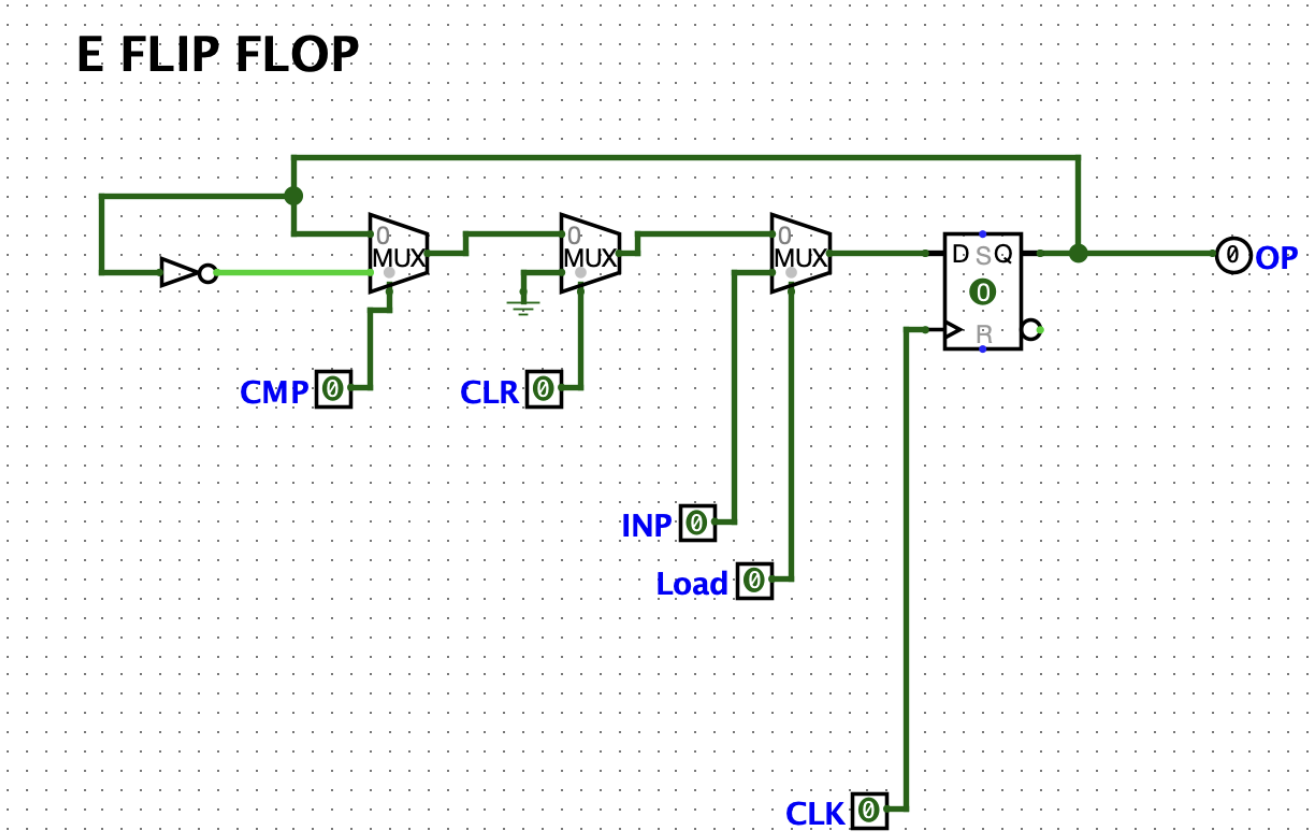



----- AC LOGIC -----
AC LOAD
AC CLR
AC INR
D0
T5
D1
T5
D2
T5
rB9
rB7
rB6
AND_OUT
ADD
DR
AC_LD
rB11
AC_CLR
rB5
AC_INC
----- IR LOGIC -----
IR LOAD
IR CLR
IR INR
T1
IR_LD
IR_CLR
0
IR_CLR
IR_INR
0
IR_INC
----- TR LOGIC -----
TR LOAD
TR CLR
TR INR
TR_LD
0
TR_LD
TR_CLR
0
TR_CLR
TR_INR
0
TR_INC



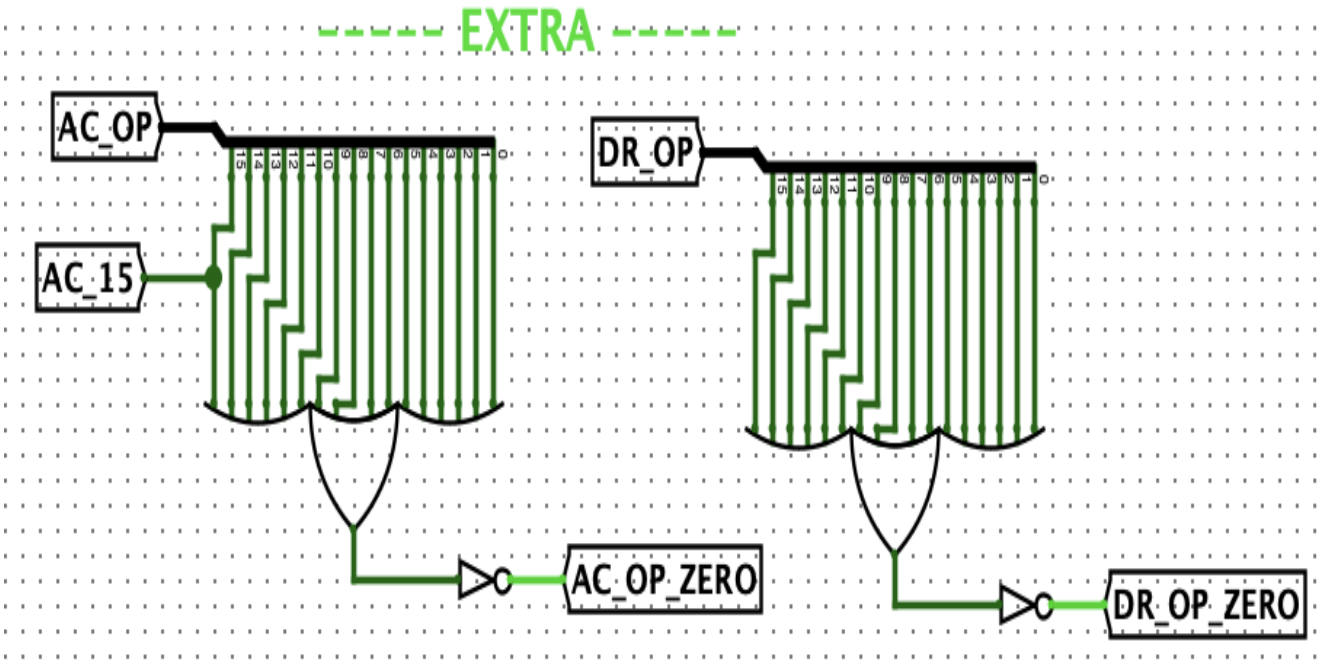
Circuit-13: Input-Output Logic.

14. E Flip Flop:



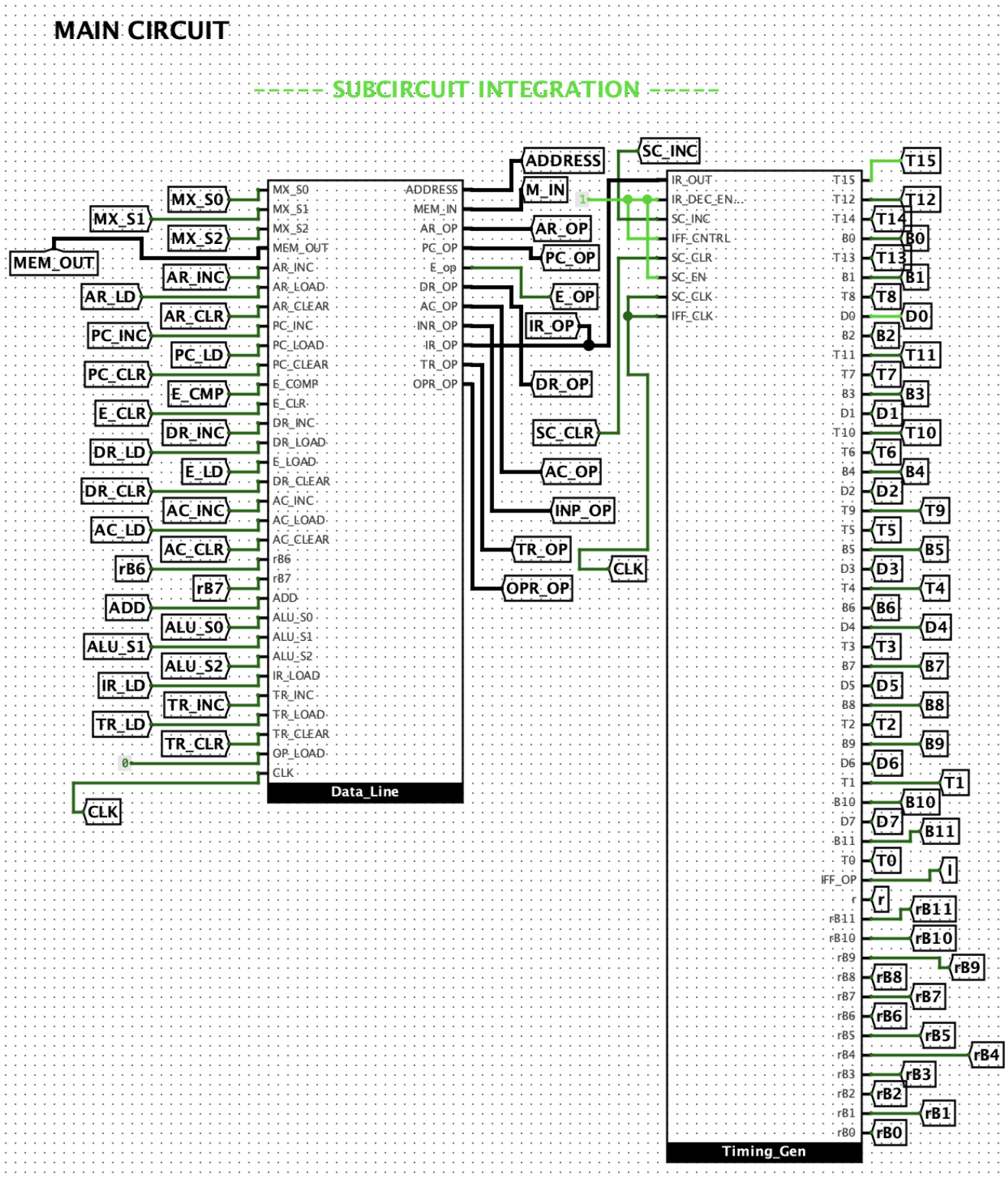
Circuit-14: E Flip Flop

15. Extra Circuit :



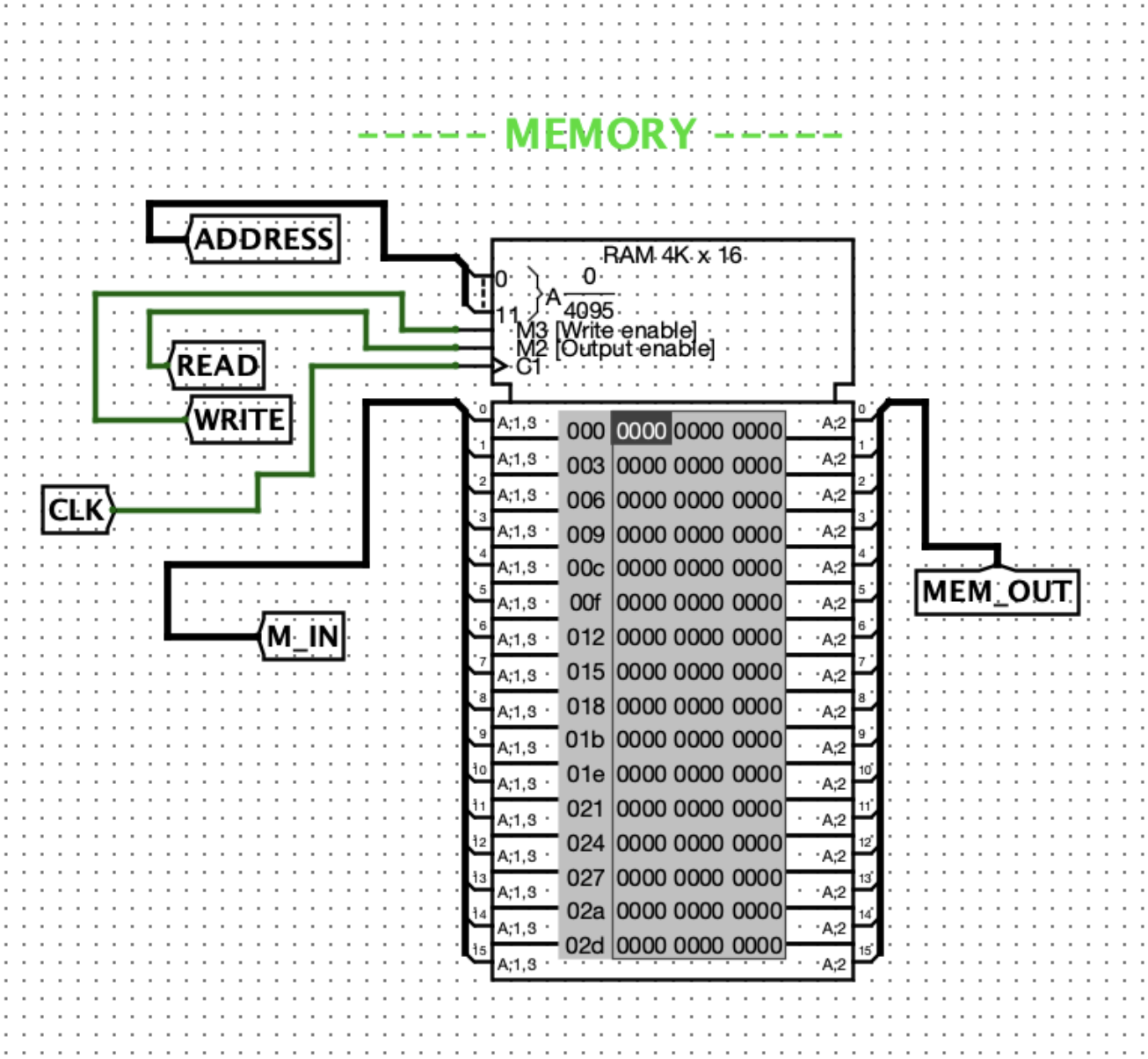
Circuit-15: Extra Circuit

16. Main Circuit:



Circuit-16: Main Circuit

17. Memory :



Circuit-17: Memory

Conclusion

We have successfully designed and implemented a basic CPU model based on the architecture described in Morris Mano's book. The design includes key components such as:

- Control Unit
- Arithmetic Logic Unit (ALU)
- Accumulator and General-Purpose Registers
- Instruction Register (IR)
- Program Counter (PC)
- Memory Address Register (AR)
- Input/Output Registers
- Bus System for Data Transfer
- Clock Pulse Generator

By simulating and analyzing the working of this CPU, we learnt :

- The execution flow of memory-reference, register-reference, and I/O instructions
- Understanding the fetch-decode-execute cycle
- Realizing how control signals trigger micro-operations

- Managing the timing of operations using the sequence counter (SC)
- Applying register transfer language (RTL) to design CPU behavior
- Learning about the binary encoding of instructions and their impact on hardware-level execution

This Project helped us understand how instructions are fetched from memory, decoded, and executed step by step. We also understood how the control unit interacts with various registers and memory to perform a specific task. Overall, this project provided us with valuable insights into the internal organization and functioning of a simple computer system